



APPLICATION OF CONVOLUTIONAL NETWORKS VIA BACKPROPAGATION

Institute of Embedded Systems
Hamburg University of Technology

Bachelor Thesis

Hani Alassiri Alhabboub
21760484

03. September 2021

First Examiner: Prof. Dr. Dr. habil. Karl-Heinz Zimmermann
Second Examiner: M. Sc. Merve Cakir

Statutory declaration

I declare that I have authored this thesis independently, that I have not used other than the declared sources / resources, and that I have explicitly marked all material which has been quoted either literally or by content from the used sources.

Date: _____ Signature: _____

Abstract

Artificial neural networks are in the center of attention nowadays with new research papers coming out about them almost every few weeks. In this work, we highlight a branch of neural networks that is also in the center of hyper-attention: convolutional neural networks.

We focus here on a couple of fundamental questions, more or less in the following order: What is a deep neural network? How is a network trained? What is the stochastic gradient method? What are convolution and convolutional neural networks? How can convolutional neural networks be applied in image classification? We answer the last question in a form of a ready to use implementation in Python. We also tweak our implementation a little bit to try to see and understand how can that influence the performance.

Contents

1	Outline	6
2	Motivation	7
3	Mathematical Foundations	8
3.1	Linear algebra formulation	8
3.2	Activation functions	9
3.3	Artificial neural networks	11
3.4	Feedforwards neural networks	12
3.4.1	Feedforward	12
3.4.2	Cost function and gradient descent	13
3.4.3	Stochastic gradient descent	16
3.4.4	Backpropagation	16
3.4.5	Problems of backpropagation and alternatives	19
4	Convolutional Neural Networks (CNNs)	21
4.1	Convolutions and Cross-Correlations	21
4.1.1	Convolution Refinements	25
4.2	CNNs	26
4.2.1	Pooling	29
4.2.2	Backpropagation in CNNs	29
5	Implementation	31
5.1	MNIST handwritten digit dataset	31
5.2	Network training and testing	31
5.3	Results	34
6	Conclusion and Future Work	36
6.1	Summary	36

List of Figures

3.1	Heaviside function [1]	9
3.2	Labeled data points in $\mathbb{R}^2$. Circles denote points in category A. Crosses denote points in category B. [2]	10
3.3	Heaviside function (red), logistic (sigmoidal) function (blue), and logistic (sigmoidal) functions with temperature parameter (black) [3]	10
3.4	A simple neural network [3]	11
3.5	Visualization of output from an artificial neural network applied to the data [2]	14
3.6	GD with different α values [3]	15
4.1	Audio signal (blue) [3]	22
4.2	2D convolution/cross-correlation [4]	24
4.3	Fully connected layer [3]	26
4.4	Convolutional layer [3]	26
4.5	Convolution arithmetic [4]	27
4.6	Validation classification by AlexNet: The upper pictures (<i>mite</i> , <i>container ship</i> , <i>motor scooter</i> , and <i>leopard</i>) were classified correctly while the lower ones (<i>grille</i> , <i>mushroom</i> , <i>cherry</i> , and <i>Madagascar cat</i>) were not [5]	28
5.1	Plot of a Subset of Images From the MNIST Dataset	32
5.2	Loss and accuracy learning curves for the model during k-Fold cross-validation	35
5.3	Three wrongly classified images from the MNIST dataset. They are in order: 4, 5 and 6	35

1 Outline

Thesis: Convolutional neural networks are a special type of artificial neural networks that are optimized for image classification and can be used to recognise hand-written images from the MNIST data set.

In chapter 3 mathematical foundations and formulations of artificial neural networks will be presented. In sections 1 - 3 we give the definitions of mathematical neurons and activation functions. Having done that we go on to define artificial neural networks (ANNs) in section 4 and in the process define the cost function and the (stochastic) gradient descent learning method in subsections 3.4.1 to 3.4.3 as a basis for explaining backpropagation in subsection 3.4.4 and its alternatives in subsection 3.4.5

In chapter 4 convolutional neural networks are defined. In section 1 convolution and cross-correlation are introduced along with some refinements to them such as padding and striding. In section 2 we go through the history of CNNs and highlight some of their successes. In subsections 4.2.1 and 4.2.2 the close companion of convolution pooling is introduced and the differences of backpropagation in CNNs vs in Feedforward neural networks is explained.

Finally in Chapter 5 a step-by-step walk-through of the code and its performance is given after the explaining the MNIST dataset of handwritten digit.

In the end a short recap of the thesis work is presented.

2 Motivation

In a world increasingly reliant on technology there is an ever increasing need to develop systems that can understand and interpret images correctly. This can range from the need of object detection in self-driving cars [6] to tumor detection in cancer patients [7]. The ability of AI to correctly label image-data is also a major step towards general AI and convolutional neural networks (CNNs) are here to help.

First thought of in the early 1940s, artificial neural networks (ANNs) came slowly but surely to dominate the field of AI, and particularly in machine learning. They have been invented by McCulloch and Pitts [8] and have developed greatly since. They are based on a mathematical abstraction of a biological neuron as a logical device. These artificial neurons are then grouped into different layers with connections between them. The network then trains itself by tuning its parameters using a huge number of training samples. The number of parameters in such a typical neural network grows rapidly with the increase in the number of layers. This can make training a model computationally heavy (and sometimes not feasible). Since in image classification each pixel is considered as a feature, images usually have a high dimensionality, which makes it necessary to increase the number of layers. This is one of many reasons why CNNs, a special kind of ANNs, are quite useful here. Both the time taken for tuning the parameters as well as the dimensionality diminishes when using CNNs without losing on the quality of models.

Dimensionality reduction is achieved using convolution, which is like having a sliding window/filter with a size less than that of the input matrix doing computations on part of that input. This window then keeps shifting left to right and top to bottom to cover the complete image. Usually the first layers of a CNN have multiple convolutional filters working and scanning the complete feature matrix and carry out the dimensionality reduction. This enables the CNN to be a very apt and fit network for image classifications and processing. In this work, we go over the foundations of artificial neural networks and build on that to present the magic powers of CNNs. We later on explore an implementation of a CNN in python and analyze its performance.

3 Mathematical Foundations

3.1 Linear algebra formulation

Before we dive into convolutional neural networks we first have to explain some mathematical formulations. To start, we define a mathematical neuron in its simplest form the following way:

A *mathematical neuron* is a device that uses a (weighted) sum of inputs to decide, depending on a threshold, to output either a zero (no activation) or a one (activation) [3].

To unpack this, consider that we have a vector of *input variables* $\mathbf{x} = \{x_1, \dots, x_n\}$. Each one of those inputs has a multiplier that we call *weight* and denote the vector of which by $\mathbf{w} = \{w_1, \dots, w_n\}$. The sum of the multiplication of inputs with their corresponding weights is then compared to the value of the threshold θ to produce either a one, if the weighted sum exceeds the threshold, or a zero, if not.

A rigorous mathematical definition is given by the following:

Let $\mathbf{x} \in \mathbb{R}^n$ be an *input vector*, then a *neuron* is a device that computes the affine linear function

$$l(\mathbf{x}) = \mathbf{w}^T \mathbf{x} - \theta, \quad l : \mathbb{R}^n \rightarrow \mathbb{R}.$$

and outputs a one if the later is positive, zero otherwise. Where $\mathbf{w} \in \mathbb{R}^n$ is the *weight vector* and $\theta \in \mathbb{R}$ is the *threshold*. Neurons are stacked on top of each other to define a *layer*. All neurons in such a layer obtain the same input $\mathbf{x} \in \mathbb{R}^n$, but the parameters (*weights and threshold*) depend on the neuron. With a layer consisting of m neurons, we have m linear functions

$$l_j(\mathbf{x}) = \mathbf{w}_j^T \mathbf{x} - \theta_j, \quad j = 1, \dots, m.$$

We want to express the m equations in a matrix form to make things easier. For that we define the *weight matrix* $\mathbf{W} \in \mathbb{R}^{m \times n}$ and *bias* $\mathbf{b} \in \mathbb{R}^m$ by:

$$\mathbf{W} := \begin{pmatrix} \mathbf{w}_1^T \\ \vdots \\ \mathbf{w}_m^T \end{pmatrix} \in \mathbb{R}^{m \times n}, \quad \mathbf{b} := \begin{pmatrix} -\theta_1 \\ \vdots \\ -\theta_m \end{pmatrix} \in \mathbb{R}^m. \quad (3.1)$$

This allows us to define $L(\mathbf{x}) := (l_1(\mathbf{x}) \dots l_m(\mathbf{x}))^T$ by:

$$L(\mathbf{x}) = \mathbf{W}^T \mathbf{x} + \mathbf{b}, \quad L : \mathbb{R}^n \rightarrow \mathbb{R}^m. \quad (3.2)$$

3.2 Activation functions

The *activation function* of a neuron defines the output (activation) of that neuron given an input or set of inputs [2]. For example, we previously said that a neuron j would output a one if the linear function $l_j(\mathbf{x})$ was positive and zero otherwise. This can be realized by a so called Heaviside or step function (Figure 3.1) defined as:

$$H(\mathbf{w}^T \mathbf{x} + b), \quad H(x) = \begin{cases} 0, & x < 0. \\ 1, & x \geq 0. \end{cases} \quad (3.3)$$

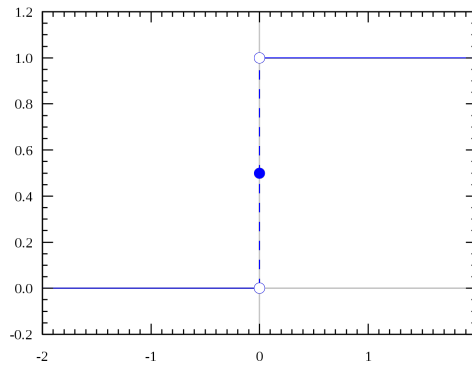


Figure 3.1: Heaviside function [1]

We can use activation functions like this one to help us with classification problems. Say we have to distinguish between two groups A and B like in Figure (3.2). The data shows some points in category A, indicated by circles, and some in category B, indicated by crosses. For example, the data may show oil drilling sites on a map, where category A denotes a successful outcome. Can we use this data to categorize a newly proposed drilling site? Our job is to construct a mapping that takes any point in $\mathbb{R}^2$ and returns either a circle (one) or a cross (zero). Of course, there are a lot of ways to construct such a mapping. The artificial neural network approach uses repeated application of a simple, nonlinear function [2]. We will elaborate on this later on.

Although the Heaviside function can be quite useful it's still has a couple of problems. For example if we were on the separating line we would always output a circle, which is unrealistic. Also, for reasons we will explain later, we want to be able to differentiate the activation function, which is not possible at point $x = 0$. So what should we do? Well we would like to have a function with (almost) the same shape, meaning it should converge to one as $x \rightarrow \infty$ and to zero as $x \rightarrow -\infty$ while being continuous in between. For that we introduce the sigmoid¹ function

$$\sigma(x) = (1 + e^{-x})^{-1}, \quad \sigma : \mathbb{R} \rightarrow (0, 1). \quad (3.4)$$

¹The sigmoid function is sometimes referred to as the logistic function

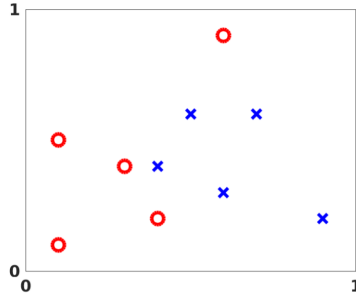


Figure 3.2: Labeled data points in $\mathbb{R}^2$. Circles denote points in category A. Crosses denote points in category B. [2]

This function varies smoothly and monotonically between zero and one. Turning back to our example, the closer the return value is to one, the more we are willing to identify the data point as belonging to a circle. This function also comes with a convenient property that its derivative takes the simple form

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)). \quad (3.5)$$

which is easy to verify. This function can be slightly adjusted by multiplying the input x with a so called temperature parameter $k > 0$. You can see the plots of all mentioned activation functions in Figure (3.3). We say that the parameter k controls the *sharpness* of the resulting curve and observe that twisting it makes the sigmoid function more Heaviside-like, it is easy to prove that in the pointwise limit $k \rightarrow \infty$ we obtain the Heaviside function. There are many other activation functions like the sign, hyperbolic tangent and ReLU that we are not going to cover here since we are going to stick with the sigmoid function. We turn now to *Artificial neural networks* and explain how all the different components we have covered so far add up to produce a computational system that can correctly classify data.

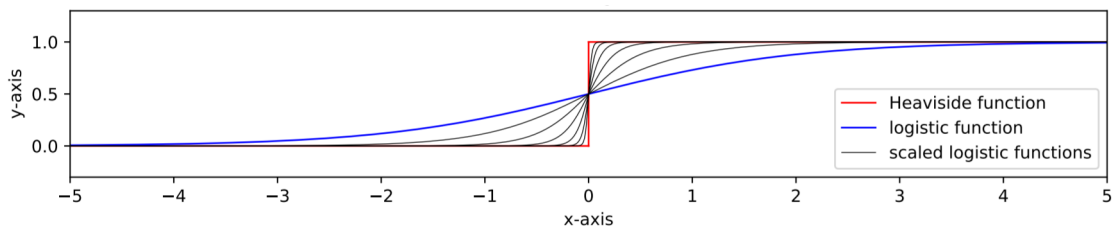


Figure 3.3: Heaviside function (red), logistic (sigmoidal) function (blue), and logistic (sigmoidal) functions with temperature parameter (black) [3]

3.3 Artificial neural networks

Artificial neural networks (ANNs), usually simply called neural networks, are computing system vaguely inspired by the biological neural networks that constitute animal brains. They were first thought of by McCulloch and Pitts [8] back in 1943. McCulloch and Pitts, however, did not consider learning, they derived a technique that allows finding a neural net that models a given Boolean expression. A model for the learning in biological neural networks was stated by Hebb [9] in the late 1940s, which is usually abbreviated to “Cells that fire together wire together”. ANNs then went through a series of changes from Kleene (1956) changing McCulloch and Pitts formulation to make the presentation much more accessible [10] to Rosenblatt’s invention of the perceptron (1958) [11] to undergoing the first AI winter (1970s-1980s) and the second (1990s-2010s) towards currently, where we are (still) in a period of rising interest in ML/AI. Due to the use of new hardware like graphics processing units (GPU) and the vast amount of data collected, e.g., by companies like Google and Facebook, the old techniques gave new, better results. This changed even more when instead of using one hidden layer it became possible to use more and more layers, i.e., a deep neural network, and other types of layers. In short, an ANN is based on a set of connected artificial neurons. Each connection, like the synapses in an animal brain, can transmit a “signal” (some real number) to other neurons. A neuron that receives a signal then processes it and can signal neurons connected to it. The output of each neuron is computed by some non-linear function of the sum of its inputs. Connections have a weight that increases or decreases the strength of the signal at a connection and keeps adjusting as the learning proceeds. Neurons have a threshold such that a signal is sent only if the aggregate signal crosses that threshold. Neurons are aggregated into layers. Different layers may perform different transformations on their inputs. Signals travel from the first layer (the input layer), to the last layer (the output layer), possibly after traversing the layers back and forth multiple times.

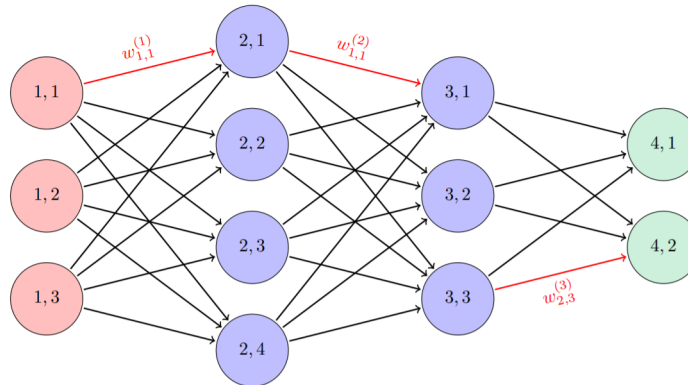


Figure 3.4: A simple neural network [3]

In Figure 3.4 we see an example of a simple neural network consisting of one input layer (represented by three red circles), two hidden layers (purple) and one output layer (green), it is worth mentioning that there is no special meaning to the word hidden; it simply indicates

that these neurons are performing intermediate calculations. Deep learning is a loosely defined term which implies that many hidden layers are being used. Notation-wise we use a pair of coordinates to indicate the location of each neuron in the network where the first number indicates the layer in which the neuron is, and the second indicates its position within that layer. When it comes to the *weights* we use the following notation: For example $w_{2,3}^{(3)}$ would be the weight of the connection between the third neuron in the third layer with the second neuron in the forth layer as indicated in Figure 3.4 and is the (2,3) entry in the third weight matrix² $\mathbf{W}_3 \in \mathbb{R}^{2 \times 3}$. This ANN represents a function $\mathbf{f}: \mathbb{R}^3 \rightarrow \mathbb{R}^2$ with $\mathbf{x} \in \mathbb{R}^3$ as input going into the three red circles and $\mathbf{f}(\mathbf{x}) \in \mathbb{R}^2$ as an output coming out of the two green circles. The function $\mathbf{f}(\mathbf{x})$ is evaluated in a loop of computation. We define:

$$\mathbf{a}_1 := \mathbf{x}, \mathbf{z}_i = \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), i = 1, \dots, 3, \mathbf{f}(\mathbf{x}) := \mathbf{a}_4. \quad (3.6)$$

where the edges of the network correspond to entries in the *weight matrix* $\mathbf{W}_i$, a_i , for $i = 1, \dots, 3$ are some of the activation functions from last section³, $\mathbf{b}_i$ is the vector of biases for layer i , and $\mathbf{a}_i$ is the vector of activations for layer i and contains the *activated* affine linear $\mathbf{z}_i$.

3.4 Feedforwards neural networks

3.4.1 Feedforward

Feedforwards neural networks are the only type of networks that we encountered so far. The name feedforward comes from the order of evaluation, meaning that activations are calculated one layer at a time starting from the first layer on to the next one and so on. This is not the most generic type of neural networks as there many other ones such as CNNs, RNN, ResNet, and Neural ODEs. In this work we are only going to explore CNNs while briefly touching upon FNNs to explain some basic concepts such as the *Cost function*, *Gradient descent* and *Backpropagation*.

Let $n \in \mathbb{N}^m$, m is the number of layers. We define a *feedforwards neural network* as a neural network where the evaluation of $\mathbf{f}: \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_m}$ happens layer-wise through the network as given in Equation 3.6:

$$\mathbf{a}_1 := \mathbf{x}, \mathbf{z}_i = \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), i = 1, \dots, 3, \mathbf{f}(\mathbf{x}) := \mathbf{a}_m.$$

and where every neuron in layer i is connected to every neuron in layer $i + 1$ with no connections within each layer or with a neuron to itself [3].

²As a reminder $\mathbf{W} := \begin{pmatrix} \mathbf{w}_1^T \\ \vdots \\ \mathbf{w}_m^T \end{pmatrix}$

³Those are commonly fixed in each layer, but may differ between layers.

3.4.2 Cost function and gradient descent

At this point you might be wondering: This seems nice and all, but how does it all work? How can such an ANN correctly predict anything? Well, at first it does not. The weights and biases are set at random and as such the network makes random predictions. The way to correct this is by training the network with large sets of data, and adjusting the weights and biases each time the network makes a wrong prediction such that it can make a correct one if given the same input again.

Back to our example in Figure 3.2, remember that our aim was to make $\mathbf{f}(\mathbf{x})$ as close to $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ as possible for data entries that belong to category **A**, and as close to $\begin{pmatrix} 0 \\ 1 \end{pmatrix}$ as possible for data entries belonging to category **B**. So it makes sense to classify any new data point according to the largest component of $\mathbf{f}(\mathbf{x})$. That is, classify a new point $\mathbf{x} \in \mathbb{R}^2$ as belonging to category **A** if $\mathbf{f}_1(\mathbf{x}) > \mathbf{f}_2(\mathbf{x})$ or to category **B** if $\mathbf{f}_1(\mathbf{x}) < \mathbf{f}_2(\mathbf{x})$, with some rule to break ties. To do that we define a so called *Cost* function

$$Cost(\mathbf{W}, \mathbf{b}) := \frac{1}{2N} \sum_{i=1}^N \|\mathbf{f}_i(\mathbf{x}) - \mathbf{y}_i\|_2^2 \quad (3.7)$$

where $\mathbf{y}_i = \mathbf{y}_i(\mathbf{x})$ is the desired or target output, N is the number of training data pairs and $\mathbf{W}, \mathbf{b}$ are the parameters that we want to optimize over such that:

$$\mathbf{f}_i(\mathbf{x}) \sim \mathbf{y}_i, \quad j = 1, \dots, N. \quad (3.8)$$

i.e. such that the *Cost* is minimal. Here, the factor $1/2$ is included for convenience; it simplifies matters when we start differentiating. This function measures the squared Euclidean distance between the outcome of the neural network and the desired output for every data pair and averages by the number of data pairs. We emphasize that *Cost* is a function of the weights and biases—the data points are fixed [2]. To solve this optimization problem we can apply for example the nonlinear least-squares solver **MATLAB** function **lsqnonlin** on the data points presented in Figure 3.2. Figure 3.5 shows the boundary where $\mathbf{f}_1(\mathbf{x}) > \mathbf{f}_2(\mathbf{x})$ for the trained network. We cannot guarantee, however, that **lsqnonlin** can always find the global minimum of a nonconvex function since we cannot exhaustively search over the entire parameter space. We need to use a different approach. For that we introduce the method of *steepest descent* or *gradient descent* which surfs as a numerical approximation to the optimization problem. It works the following way:

Let $f : \mathbb{R} \rightarrow \mathbb{R}$. Suppose⁴ (piecewise) $f \in C^1$. We start at a random input point $x \in \mathbb{R}$ and try to figure out in which direction should we step in order to reduce the output $f(x)$. To do that we calculate the derivative in x , which is the slope. If the derivative is positive we proceed in negative direction and if derivative is negative we proceed in positive direction. We say, that if the derivative is continuous and nonzero, then there exists some $\lambda > 0$, such that for $\lambda > \alpha > 0$:

$$f(x - f'(x)\alpha) < f(x).$$

⁴ $f \in C^1$ means that f is at least one time continuously differentiable

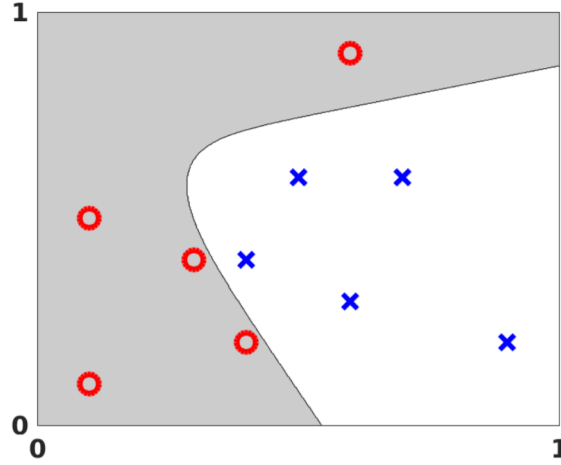


Figure 3.5: Visualization of output from an artificial neural network applied to the data [2]

This technique is used iteratively. So we choose some appropriate starting value $x_0 \in \mathbb{R}$ and then iterate

$$x_{k+1} := x_k - f'(x_k)\alpha, \quad k \in \mathbb{N}_0. \quad (3.9)$$

Here α is a small step size that can be tuned to either accelerate or slow down the learning process. This equation defines the steepest descent method. We choose an initial vector and iterate with Equation (3.9) until some stopping criterion has been met, or until the number of iterations has exceeded the computational budget. Of course, we would want to generalize this to a multi-dimensional case where the function depends on multiple variants. So how does a multi-dimensional derivative look like? Consider a real function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that is at least once continuously differentiable. The *derivative* $Df : \mathbb{R}^n \rightarrow \mathbb{R}$ is a linear map. This implies that the derivative of such a function f at a point $\mathbf{x} \in \mathbb{R}^n$ is a row vector. The transposed object, i.e., here a column vector, is termed the *gradient* of f , $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}$. The derivative and the gradient are defined in terms of the partial derivatives of f with respect to the variable x_i , $i = 1, \dots, n$

$$Df_{\mathbf{x}} = \left(\frac{\partial f}{\partial x_1}(\mathbf{x}) \quad \dots \quad \frac{\partial f}{\partial x_n}(\mathbf{x}) \right), \quad \nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{pmatrix} = (Df_{\mathbf{x}})^T \quad (3.10)$$

This is where the name *gradient descent* comes from. To derive the method we assume that f is twice continuously differentiable at $\mathbf{x}_0$ and use the Taylor expansion at $\mathbf{x}_0$

$$f(\mathbf{x}) = f(\mathbf{x}_0) + Df(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2) = f(\mathbf{x}_0) + \langle \nabla f(\mathbf{x}_0), \mathbf{x} - \mathbf{x}_0 \rangle + O(\|\mathbf{x} - \mathbf{x}_0\|_2^2). \quad (3.11)$$

Now we have to find the direction of the *steepest descent*. We can address this via the *Cauchy-Schwarz* inequality, which states that for any $u, v \in \mathbb{R}^n$:

$$- \|u\|_2 \|v\|_2 \leq \langle u, v \rangle \leq \|u\|_2 \|v\|_2.$$

So the most negative that $\langle u, v \rangle$ can get is $-||u||_2 ||v||_2$ which happens to occur when $u = -v$. So how does this help? Going back to Equation 3.11 we see that to reduce the value of $f(\mathbf{x})$ we have to subtract something from $f(\mathbf{x}_0)$. Let us ignore the term $O(||\mathbf{x} - \mathbf{x}_0||_2^2)$ for now and concentrate on making the term $\langle \nabla f(\mathbf{x}_0), x - \mathbf{x}_0 \rangle$ negative. We do this by setting a new point $\mathbf{x}_1$ such that $\mathbf{x}_1 - \mathbf{x}_0 = -\alpha \nabla f(\mathbf{x}_0)$, and thus obtain

$$f(\mathbf{x}_1) = f(\mathbf{x}_0) - \alpha ||\nabla f(\mathbf{x}_0)||_2^2 + O(\alpha^2 ||\nabla f(\mathbf{x}_0)||_2^2).$$

The term $O(\alpha^2 ||\nabla f(\mathbf{x}_0)||_2^2)$ can be neglected for a sufficiently small stepsize $\alpha > 0$. Thus we obtain with the *negative gradient* the direction of steepest descent needed for our method. So for $k \in \mathbb{N}_0$

$$f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k) - \alpha ||\nabla f(\mathbf{x}_k)||_2^2 + O(\alpha^2 ||\nabla f(\mathbf{x}_k)||_2^2). \quad (3.12)$$

and

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k), \quad \mathbf{x}_0 \in \mathbb{R}^n. \quad (3.13)$$

It is worth mentioning that the *step size* α can either vary for each step, which is theoretically ideal, or be a constant, which is what actually happens in practise as we do not have sufficient information about the function. Figure 3.6 shows a plot of four GDs with different α values working their way around towards getting to the minimum. They all start at the same point x_0 and then take exactly 10 steps. We observe that for a very small α any improvements will possibly be very small too and that for α too big there will possibly be no improvements at all as the GD jumps back and forth. In the shown example the step size $\alpha = 1/8$ seems to be optimal. In practise, multiple α values need to be experimented with before finding out the best one. We now tweak our notation to fit the machine learning

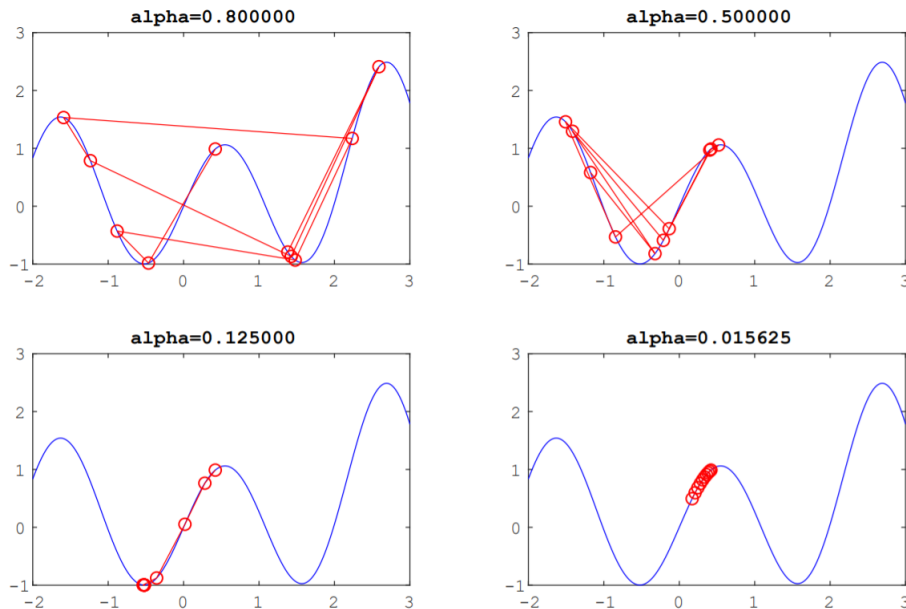


Figure 3.6: GD with different α values [3]

model. The weights and biases take the form of matrices and vectors, but at this stage it is convenient to imagine them stored as a single vector that we call $p \in \mathbb{R}^s, s \in \mathbb{N}$. let $\mathbf{C} : \mathbb{R}^s \rightarrow \mathbb{R}$ be the *Cost* function as defined in Equation (3.7) and η be the step size that we will from now on start calling the *learning rate*. We update by:

$$p \leftarrow p - \nabla \mathbf{C}(p) \eta \quad (3.14)$$

3.4.3 Stochastic gradient descent

The calculation of $\nabla \mathbf{C}(p)$ depends on calculating a relatively huge number of partial derivatives. There exists some ways to reduce the number partial derivatives needed like *Stochastic gradient descent*. It is a much cheaper alternative that works by replacing the mean of the individual gradients over all training points by the gradient at a single, randomly chosen, training point. A single step may be summarized as follows:

1. Let N be the number training samples. Choose an integer i randomly and uniformly, $i \in \{1, \dots, N\}$.
2. Update

$$p \leftarrow p - \nabla \mathbf{C}_i(p) \eta. \quad (3.15)$$

This may seem like a dramatic reduction, but the hope is that as the iteration proceeds, the method sees more training points and the reduction will be worthwhile overall.

There exist several alternatives. One of which is where instead of picking the index i *with replacement* we can instead sample without it; meaning we go through each of the N training points in a random order. Performing N steps in this manner, referred to as completing an *epoch*, may be summarized as follows:

1. Compute a permutation σ of $\{1, \dots, N\}$.
2. Update

$$p \leftarrow p - \nabla \mathbf{C}_{\sigma(i)}(p) \eta, \quad i = 1, \dots, N. \quad (3.16)$$

Another alternative is to just use a small batch, called *minibatch*, of the entire training set and picking a new batch each epoch.

3.4.4 Backpropagation

We have seen that we can find an optimum, say a minimum, using (stochastic) gradient descent. Which as a reminder needs the derivatives of the cost function with respect to the parameters, i.e., the weight matrices and biases. The computation of these is based on the so-called *backpropagation*, a clever utilization of the multivariate chain rule. The term was announced and then popularized by Rumelhart [12], Hinton & Williams (1986), but the technique was independently rediscovered many times, and had many predecessors dating to the 1960s: In 1961, the basics concept of continuous backpropagation were derived in the context of control theory by J. Kelly, Henry Arthur, and E. Bryson [13]. In 1969, Bryson and Ho gave a multi-stage dynamic system optimization method [14]. In 1974, Werbos stated the

possibility of applying this principle in an artificial neural network. In 1986, by the effort of David E. Rumelhart, Geoffrey E. Hinton, Ronald J. Williams, backpropagation gained recognition [12]. And finally in 1993, Wan was the first person to win an international pattern recognition contest with the help of the backpropagation method. To explain backpropagation we first switch from the general vector of parameters, p , used in the previous section to the entries in the weight matrices and bias vectors. Remember, our task is to compute partial derivatives of the cost function with respect to each $\mathbf{W}$ and $\mathbf{b}$. We therefore focus our attention on computing those individual partial derivatives. As a short recap we state the multivariate chain rule. Let $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^k$ be continuously differentiable. The *Jacobian* of $\mathbf{g}$, i.e., the first derivative of $\mathbf{g}$, is a matrix function $\mathbf{J}_{\mathbf{g}} : \mathbb{R}^n \rightarrow \mathbb{R}^{k \times n}$ given by

$$\mathbf{J}_{\mathbf{g}}(\mathbf{x}) := D\mathbf{g}(\mathbf{x}) = \begin{pmatrix} \frac{\partial g_1}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial g_1}{\partial x_n}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \frac{\partial g_k}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial g_k}{\partial x_n}(\mathbf{x}) \end{pmatrix} \in \mathbb{R}^{k \times n} \quad \forall \mathbf{x} \in \mathbb{R}^n. \quad (3.17)$$

Now let the composition $\mathbf{h} := \mathbf{f} \circ \mathbf{g}$, $\mathbf{h} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ defined by $\mathbf{h}(\mathbf{x}) = \mathbf{f}(\mathbf{g}(\mathbf{x}))$ for all $\mathbf{x} \in \mathbb{R}^n$ of two continuously differentiable functions $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^k$ and $\mathbf{f} : \mathbb{R}^k \rightarrow \mathbb{R}^m$ be given. Then

$$\mathbf{h}(\mathbf{x}) = \mathbf{f}(\mathbf{g}(\mathbf{x})), \quad \mathbf{y} := \mathbf{g}(\mathbf{x}) \Rightarrow \mathbf{J}_{\mathbf{h}}(\mathbf{x}) = \mathbf{J}_{\mathbf{f}}(\mathbf{y}) \mathbf{J}_{\mathbf{g}}(\mathbf{x}) \quad \forall \mathbf{x} \in \mathbb{R}^n.$$

Note that in Equation (3.10) we mentioned that the difference between the derivative and the gradient amounts to transposition of the objects. This means when working with transposed derivatives to obtain the gradients we intend to use in SGD, the order of the matrix products in the chain rule is reversed. Let us turn now to our first theorem.

Theorem 3.4.1 (Partial derivatives of an FNN.). *Let an FNN $f : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_m}$,*

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i = \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m.$$

for the weight matrices, biases, and activation functions

$$\mathbf{W}_i \in \mathbb{R}^{n_{i+1} \times n_i}, \quad \mathbf{b}_i \in \mathbb{R}^{n_{i+1}}, \quad a_i : \mathbb{R} \rightarrow \mathbb{R}, \quad i = 1, \dots, m-1$$

respectively, and the local cost function

$$\mathbf{C}(\mathbf{p}) = \frac{1}{2} \|\mathbf{f}_i(\mathbf{x}) - \mathbf{y}_i\|_2^2.$$

be given. Define for $i = 1, \dots, m-1$ the vectors

$$\delta_i := \frac{\partial \mathbf{C}}{\partial \mathbf{z}_i} := \begin{pmatrix} \frac{\partial \mathbf{C}}{\partial z_{1,i}} \\ \vdots \\ \frac{\partial \mathbf{C}}{\partial z_{i+1,i}} \end{pmatrix} = \nabla_{\mathbf{z}_i} \mathbf{C}.$$

The vectors δ_i can be computed recursively from the last layer to the first,

$$\delta_{m-1} = a'_{m-1}(\mathbf{z}_{m-1}) \circ (\mathbf{a}_m - \mathbf{y}), \quad \delta_i = a'_i(\mathbf{z}_i) \circ (\mathbf{W}_{i+1}^T \delta_{i+1}), \quad i = m-2, m-1, \dots, 1.$$

The derivatives of the cost function $\mathbf{C}$ with respect to the weights $\mathbf{W}_i$ and the biases $\mathbf{b}_i$, $i = 1, \dots, m-1$, are then given in terms of the δ -vectors by

$$\frac{\partial \mathbf{C}}{\partial \mathbf{W}_i} = \delta_i \mathbf{a}_i^T, \quad \frac{\partial \mathbf{C}}{\partial \mathbf{b}_i} = \delta_i$$

Proof. The derivative of the partial cost function with respect to $\mathbf{a}_m$, interpreted as a column vector, i.e., gradient, is given by

$$\frac{\partial \mathbf{C}}{\partial \mathbf{a}_m} = \frac{1}{2} \frac{\partial \langle \mathbf{a}_m - \mathbf{y}, \mathbf{a}_m - \mathbf{y} \rangle}{\partial \mathbf{a}_m} = \frac{1}{2} \frac{\partial (\mathbf{a}_m^T \mathbf{a}_m - 2\mathbf{y}^T \mathbf{a}_m + \mathbf{y}^T \mathbf{y})}{\partial \mathbf{a}_m} = \mathbf{a}_m - \mathbf{y} = \nabla_{\mathbf{a}_m} \mathbf{C}.$$

For $i = 1, \dots, m-1$, since $\mathbf{a}_{i+1} = a_i(\mathbf{z}_i)$, $\mathbf{z}_i \in \mathbb{R}^{n_{i+1}}$,

$$\frac{\partial \mathbf{a}_{i+1}}{\partial \mathbf{z}_i} = (\mathbf{J}_{a_{i+1}}(\mathbf{z}_i))^T = \begin{pmatrix} \frac{\partial a_i(z_1^{(i)})}{\partial z_1^{(i)}} & \cdots & \frac{\partial a_i(z_{n_{i+1}}^{(i)})}{\partial z_1^{(i)}} \\ \vdots & \ddots & \vdots \\ \frac{\partial a_i(z_1^{(i)})}{\partial z_{n_{i+1}}^{(i)}} & \cdots & \frac{\partial a_i(z_{n_{i+1}}^{(i)})}{\partial z_{n_{i+1}}^{(i)}} \end{pmatrix} = \mathbf{diag}(a'_i(\mathbf{z}_i))$$

Due to the (transposed) chain rule,

$$\delta_{m-1} = \frac{\partial \mathbf{C}}{\partial \mathbf{z}_{m-1}} = \frac{\partial \mathbf{a}_m}{\partial \mathbf{z}_{m-1}} \frac{\partial \mathbf{C}}{\partial \mathbf{a}_m} = \mathbf{diag}(a'_{m-1}(\mathbf{z}_{m-1}))(\mathbf{a}_m - \mathbf{y}) = a'_{m-1}(\mathbf{z}_{m-1}) \circ (\mathbf{a}_m - \mathbf{y}),$$

where $\circ$ here denotes the *componentwise multiplication* or *Hadamard product*. That was the expression for δ_{m-1} . Now we have to compute all the other δ_i vectors recursively from the last layer to the first. We know that the affine combination $\mathbf{z}_{i+1} = \mathbf{W}_{i+1} \mathbf{a}_{i+1} + \mathbf{b}_{i+1}$ for $i = 0, \dots, m-2$ holds true and thus

$$\frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{a}_{i+1}} = (\mathbf{J}_{\mathbf{z}_{i+1}}(\mathbf{a}_{i+1}))^T = \mathbf{W}_{i+1}^T$$

we conclude, again using the (transposed) chain rule, that

$$\delta_i = \frac{\partial \mathbf{C}}{\partial \mathbf{z}_i} = \frac{\partial \mathbf{a}_{i+1}}{\partial \mathbf{z}_i} \frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{a}_{i+1}} \frac{\partial \mathbf{C}}{\partial \mathbf{z}_{i+1}} = \mathbf{diag}(a'_i(\mathbf{z}_i)) \mathbf{W}_{i+1}^T \delta_{i+1} = a'_i(\mathbf{z}_i) \circ \mathbf{W}_{i+1}^T$$

for all $i = 1, \dots, m-2$. The equation for the derivative of $\mathbf{C}$ with respect to $\mathbf{b}_i$ follows again using the (transposed) chain rule, since $\mathbf{z}_i = \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i$ and

$$\frac{\partial \mathbf{z}_i}{\partial \mathbf{b}_i} = (\mathbf{J}_{\mathbf{z}_i}(\mathbf{b}_i))^T = \mathbf{I}, \quad \text{i.e.,} \quad \frac{\partial \mathbf{C}}{\partial \mathbf{b}_i} = \frac{\partial \mathbf{z}_i}{\partial \mathbf{b}_i} \frac{\partial \mathbf{C}}{\partial \mathbf{z}_i} = \mathbf{I} \delta_i = \delta_i.$$

To compute $\frac{\partial \mathbf{C}}{\partial \mathbf{W}_i}$ we drop the indices, i.e., we use $\delta = \frac{\partial \mathbf{C}}{\partial \mathbf{z}}$ and $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$. This is only done to make the notation easier and less confusing. Since we no longer use i as index of the layer, we can use it as row index to conform to standard matrix notation. The derivative of $z_i = \mathbf{e}_i^T \mathbf{z}$ with respect to $w_{i,j} = \mathbf{e}_i^T \mathbf{W} \mathbf{e}_j$ is given⁵ by

$$\frac{\partial z_i}{\partial w_{i,j}} = \frac{\partial (\sum_k w_{i,k} a_k + b_i)}{\partial w_{i,j}} = a_j = \mathbf{a}^T \mathbf{e}_j.$$

The other derivatives $\partial z_l / \partial w_{i,j}$ for $l \neq i$ are all zero. Therefore the derivative of $\mathbf{C}$ with respect to $w_{i,j}$ and the "matrix derivative" $\partial \mathbf{C} / \partial \mathbf{W}$ are given by the univariate chain rule as

$$\frac{\partial \mathbf{C}}{\partial w_{i,j}} = \frac{\partial \mathbf{C}}{\partial z_i} \frac{\partial z_i}{\partial w_{i,j}} = \delta_i a_j = \mathbf{e}_j^T \boldsymbol{\delta} \mathbf{a}^T \mathbf{e}_j \Rightarrow \frac{\partial \mathbf{C}}{\partial \mathbf{W}} = \boldsymbol{\delta} \mathbf{a}^T.$$

Substituting the missing layer index i finishes the proof. [3] □

We recall that the output $\mathbf{a}_m$ can be evaluated from a forward pass through the network, computing $\mathbf{a}_1, \mathbf{z}_2, \mathbf{a}_2, \mathbf{z}_3, \dots, \mathbf{a}_m$ in order. Having done this, we see from our theorem that $\boldsymbol{\delta}_m$ is immediately available. Then, from $\boldsymbol{\delta}_i = \mathbf{a}'_i(\mathbf{z}_i) \circ (\mathbf{W}_{i+1}^T \boldsymbol{\delta}_{i+1})$ the rest of the $\boldsymbol{\delta}_i$ vectors may be computed in a backward pass. From $\frac{\partial \mathbf{C}}{\partial \mathbf{W}_i} = \boldsymbol{\delta}_i \mathbf{a}_i^T$ and $\frac{\partial \mathbf{C}}{\partial \mathbf{b}_i} = \boldsymbol{\delta}_i$ we then have access to the partial derivatives. Computing gradients in this way is what is known as *backpropagation*. [3]

3.4.5 Problems of backpropagation and alternatives

We have seen so far what we can accomplish via backpropagation. It does come, however, with its own set of limitations. One of the major disadvantages of the backpropagation learning rule is its ability to get stuck in local minima. Remember that the error is a function of all the weights and biases in a multidimensional space. This may be visualised in a three dimensional space as the error surface in a landscape with hills and valleys. There is no proof that the global minimum of the error surface has been reached. It can happen that backpropagation gets stuck in a local valley. Starting with different weights leads to different minima if the error surface is rugged. Thus randomizing the initial weights and biases and going through multiple iterations is usually necessary if not a must to avoid the local minima trap. It is important to consider this when analysing the final minimum. The learning is run repeatedly at different starting locations to show that the minimum is reasonable [15].

Another major drawback is that the convergence tends to be extremely slow [16]. Intuitively speaking, one would just try to increase the learning rate to make learning faster. This is problematic however due to the fact that one might just as easily overshoot the minimum. Several solutions have been proposed, with fast training algorithms such as [17] and [18] and second-order training methods like the quasi-Newton methods [19] and conjugate gradient methods [20]. One of those methods, called Quickprop (Fahlman, 1988) [16], is roughly

⁵ $\mathbf{e}_i$ is the unit vector i

based on Newton’s method and has outperformed back-propagation (with various adjustments) on problem domains like the ‘N-M-N Encoder’ task — i.e. training a de-/encoder network with N inputs, M hidden units and N outputs. Fahlman’s approach works on the assumption that the loss functions can be approximated by a simple parabola that opens in the positive direction. This way, calculating the minimum (of the parabola) is as simple as finding the intersection of a line with the x-axis. And if that point is not yet a minimum of the loss function, the next parabola can be approximated from there. This approach however also has its problems as some have pointed out that experiments [21] showed a degrading performance of Quickprop for larger networks when compared to gradient descent. To make things simpler, and since *backpropagation* is already very popular, we are going stick with it when applying Convolutional Neural Networks, which we now turn to.

4 Convolutional Neural Networks (CNNs)

It took us a long way, but now we are finally here. We have seen how a normal FNN can be used for classification problems. This type of neural networks however falls short on image classification. The first problem it faces is the ridiculously high number of dimensionality and training parameters. As we will see, the weight matrix in a CNN has a so-called Teoplitz form meaning it contains a lot of zero elements, which obviously leads to fewer parameters being trained and thus less computations. This is almost always accompanied with pooling, which is an operation that allows us to select only a portion of the input depending on some criteria. Another problem FNNs face is that spatial distribution within an image is neglected, meaning that they depend too much on the precise absolute location, and not so much on the relative locations of several data points. This is highly important, because the relative locations of data points is what makes say a cup of tea a cup of tea regardless of which perspective or angle we are looking at it from. Images, sound clips and many other similar kinds of data have an intrinsic structure. More formally, they share these important properties:

- They are stored as multi-dimensional arrays.
- They feature one or more axes for which ordering matters (e.g., width and height axes for an image, time axis for a sound clip).
- One axis, called the channel axis, is used to access different views of the data (e.g., the red, green and blue channels of a color image, or the left and right channels of a stereo audio track).

We need to view our data in light of all the different channels and of the relationship between them. FNNs typically view the channels only separately. As we will see, this is where convolution especially comes in hand. We first introduce the concept of convolution and then elaborate on its usefulness and application in a neural network model.

4.1 Convolutions and Cross-Correlations

In its most general form, convolution is an operation on two functions of a real-valued argument. It can be viewed from two different perspectives: as a sum and as a matrix operation. Let us move away from images for a second and turn to 1D (audio) signals with equidistant time $t_1, t_2 = t_1 + h, \dots$. We want to analyse the signal. More precisely we want to detect patterns of rapid changes, say detect loudness in an audio signal. The absolute location is irrelevant for us. We only want to detect the relative change in position. One idea is to use approximate derivatives. Consider for example the signal in Figure (4.1). The

signal is continuously differentiable and we only have its values at certain time points, say for example at t_i and t_{i+1} . To measure the change of the signal between those two points we compute the numerical derivative which, because we only have discrete points, is just the slope. So numerically, we compute some first order difference approximation

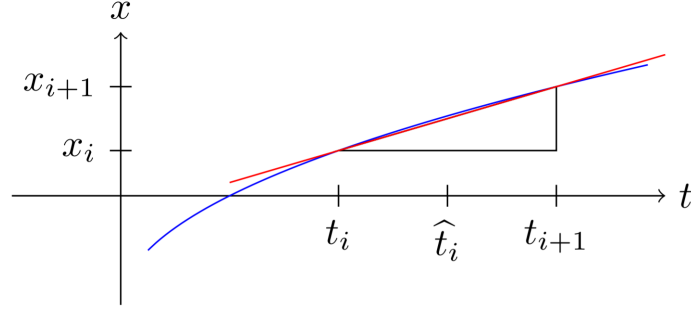


Figure 4.1: Audio signal (blue) [3]

$$x'(\hat{t}_i) \approx \frac{x_{i+1} - x_i}{t_{i+1} - t_i} = \frac{x_{i+1} - x_i}{h} = \sum_{K=0}^1 x_{i+K} d_K, \quad \mathbf{d} := \begin{pmatrix} d_0 \\ d_1 \end{pmatrix} = \begin{pmatrix} -1 \\ 1 \end{pmatrix} / h. \quad (4.1)$$

You might think that rewriting the difference approximation as a linear combination is unnecessary, but it helps us generalize later on. We state that if we apply Equation (4.1) for every pair of points in the signal $\mathbf{x}$ with the same vector $\mathbf{d}$, we obtain something which is called their *Cross-Correlation*, usually denoted by a star $\star$. It measures how similar is the signal we are looking at (in this case $\mathbf{x}$) to the some other signal (in this case $\mathbf{d}$).

$$(\mathbf{x} \star \mathbf{d})_i = \sum_{K=0}^1 x_{i+K} d_K. \quad (4.2)$$

As we have stated, the signal $\mathbf{d}$ is set to equal $\begin{pmatrix} -1 \\ 1 \end{pmatrix} / h$ meaning it has a positive slope. That is why we apply *cross-correlation* of $\mathbf{x}$ with $\mathbf{d}$, so that we actually measure how much $\mathbf{x}$ behaves like $\mathbf{d}$. *Convolution* on the other hand, denoted by an asterisk $*$, takes the other way around, meaning we numerate differently:

$$(\mathbf{x} * \mathbf{s})_{i+1} = \sum_{K=0}^1 x_{i+1-K} s_K, \quad \mathbf{s} = \begin{pmatrix} 1 \\ -1 \end{pmatrix} / h, \quad (4.3)$$

where the vectors $\mathbf{d}$ and $\mathbf{s}$ are related by just flipping the entries.

$$\mathbf{s} = \begin{pmatrix} 1 \\ -1 \end{pmatrix} / h, \quad \mathbf{d} = \begin{pmatrix} -1 \\ 1 \end{pmatrix} / h = \mathbf{F}_2 \mathbf{s} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \mathbf{s}.$$

In convolutional network terminology, the first argument (in this example, the vector $\mathbf{x}$) to the convolution is often referred to as the *input*, and the second argument (in this example,

the vector $\mathbf{d}$, or the vector $\mathbf{s}$) as the *kernel* and/or the *filter*, which is the sliding window we talked about from before. Usually the size of the *filter* is way smaller than the size of the input data. The output is sometimes referred to as the *feature map*.

Convolution is commutative, meaning we can equivalently write

$$(\mathbf{u} * \mathbf{v})_i = (\mathbf{v} * \mathbf{u})_i.$$

Let us say we want to calculate all the approximate derivatives at midpoints. The convolution sum is also equivalent to the following matrix vector multiplication.

$$\mathbf{y} = \begin{pmatrix} x'(\hat{t}_1) \\ \vdots \\ x'(\hat{t}_{n-1}) \end{pmatrix} = \begin{pmatrix} -1 & 1 & & \\ & \ddots & \ddots & \\ & & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} =: \mathbf{T} \mathbf{x}, \text{ weight } \mathbf{W} = \mathbf{T} \text{ sparse Toeplitz.}$$

where $\mathbf{T}$ has a so called Toeplitz structure. Writing convolution/cross-correlation in this way allows us to think of it as a linear operation with a structured sparse weight matrix, which will be the kind of matrix used as a weight matrix in a CNN. We remark that roles of input and kernel are interchangeable, usually the kernel here refers to the one with a lower dimension $k \ll n$. There are two kinds of convolutions that we can compute. The first one (the one we computed already) is where the *kernel* completely overlaps the data. Convolution is also defined for partial overlap. Then, however, we have to pad the input with zeros. Let $\mathbf{x} \in \mathbb{R}^n$ be the input and $\mathbf{f} \in \mathbb{R}^k$ the kernel, then the partial output (complete overlap) will be $\mathbf{y}_p \in \mathbb{R}^{n-k+1}$ and the full output (partial overlap) will be $\mathbf{y}_f \in \mathbb{R}^{n+k-1}$. So in general, for input $\mathbf{x} \in \mathbb{R}^n$ and kernel $\mathbf{f} \in \mathbb{R}^k$, we define

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_{n+k-1} \end{pmatrix} = \mathbf{x} * \mathbf{f} = \mathbf{T} \mathbf{x} := \begin{pmatrix} f_1 & & & \\ \vdots & \ddots & & \\ \hline f_k & \dots & f_1 & \\ & \ddots & \ddots & \ddots \\ & & f_k & \dots & f_1 \\ \hline & & & \ddots & \vdots \\ & & & & f_k \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_k \end{pmatrix}, \mathbf{T} \in \mathbb{R}^{(n+k-1) \times n}. \quad (4.4)$$

In the case of complete overlap only the middle part of $\mathbf{T}$ is multiplied with, while in the case where partial overlap is included the matrix $\mathbf{T}$ is taken as a whole. For a graphical representation of padding see Figure (4.5) for the 2D case. We now switch to convolution and cross-correlation in multiple dimensions to give a more general notion of them and start with 2D. In 2D, the input and output are matrices. In Figure (4.2) for example we observe that the input is a 4×4 matrix $X \in \mathbb{R}^{4 \times 4}$ (light blue). Since the input is a square matrix, we use i for both dimensions, thus $i = 4$. The output is a 2×2 Matrix $Y \in \mathbb{R}^{2 \times 2}$ (light

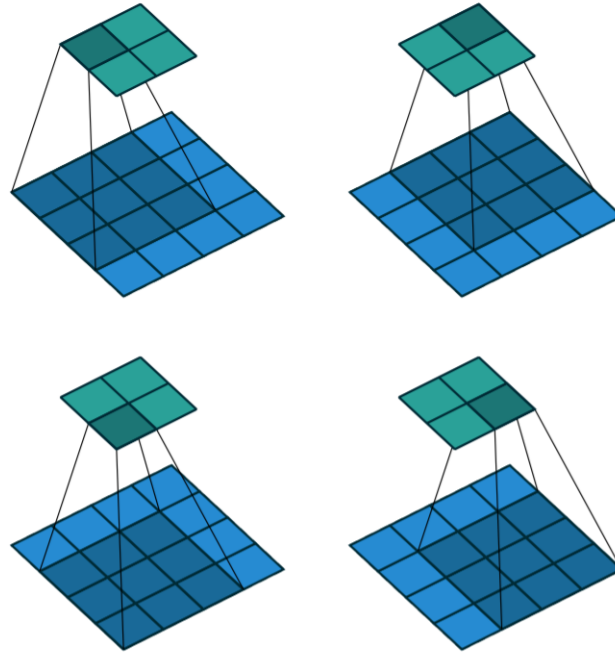


Figure 4.2: 2D convolution/cross-correlation [4]

green), likewise $o = 2$. The kernel/filter is also a matrix, here: $F \in \mathbb{R}^{3 \times 3}$, $k = 3$, which is represented by the shadow on the input that moves around at each step. We observe, since this is a full overlap, that $o = i - k + 1$.

To calculate the cross-correlation we simply overlap at each step every entry of the filter with every entry in the input matrix laying underneath it, multiply the corresponding entries with one another and then sum over all of them and obtain a single entry in the output matrix. We then shift the filter by one and repeat the process until all the elements in the input matrix have been covered.

To calculate the convolution we do the same but after flipping the kernel horizontally & vertically i.e., we rotate by 180 degrees. So in 2D *convolution* and *cross-correlation* can have the following form:

$$\mathbf{y} = \begin{pmatrix} y_{1,1} & y_{1,2} \\ y_{2,1} & y_{2,2} \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} * \begin{pmatrix} f_{1,1} & f_{1,2} \\ f_{2,1} & f_{2,2} \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} \star \begin{pmatrix} f_{2,2} & f_{2,1} \\ f_{1,2} & f_{1,1} \end{pmatrix}. \quad (4.5)$$

The dimensions of the *input* and *filter* can of course vary. Those exact ones have been chosen just to illustrate the point.

Since we would like to think of convolution/cross-correlation as a matrix vector multiplica-

tion, we can equivalently write:

$$\mathbf{y} = \begin{pmatrix} y_{1,1} \\ y_{1,2} \\ y_{2,1} \\ y_{2,2} \end{pmatrix} = \begin{pmatrix} f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 \\ 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} \end{pmatrix} \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ x_{3,1} \\ x_{3,2} \\ x_{3,3} \end{pmatrix}, \quad (4.6)$$

which is easy to verify.

Since the resulting matrix consists of minor Toeplitz matrices we call it a block Toeplitz matrix with Toeplitz blocks. We observe that the usage of such a matrix as a weight matrix in a neural network results in way fewer weights being trained. In Figures (4.3) and (4.4) one can see the difference between the number of connections in fully connected ANN ($9 \times 4 = 36$ connections) vs. a CNN ($4 \times 4 = 16$ connections). In Figure (4.4) you can also see that the connections in a convolutional layer are grouped into different colors. Each color there represents a shared connection meaning that we actually need to train only 4 different parameters. Cross-correlation and convolution can be extended to dD for any $d \in \mathbb{N}$,

$$(\mathbf{X} \star \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}_{\mathbf{i}}(n_1 + i_1, \dots, n_d + i_d) \cdot \mathbf{F}(i_1, \dots, i_d),$$

$$(\mathbf{X} * \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}_{\mathbf{i}}(n_1 - i_1, \dots, n_d - i_d) \cdot \mathbf{F}(i_1, \dots, i_d).$$

They are related by flipping the kernel in every dimension. Since multi-dimensional convolution is commutative: $\mathbf{X} * \mathbf{F} = \mathbf{F} * \mathbf{X}$

- 3D convolution can be written using a block Toeplitz matrix with blocks that are block Toeplitz with blocks that are Toeplitz.
- dD convolution can be written using a block Toeplitz matrix with blocks that are block Toeplitz with blocks that are block Toeplitz ... and so on.

4.1.1 Convolution Refinements

2D convolutions comes with different adjustments and refinements. We discuss here a couple of different ways we can pad and shift the filter. For example suppose that:

- input size: $i_1 \times i_2$,
- output size $o_1 \times o_2$,

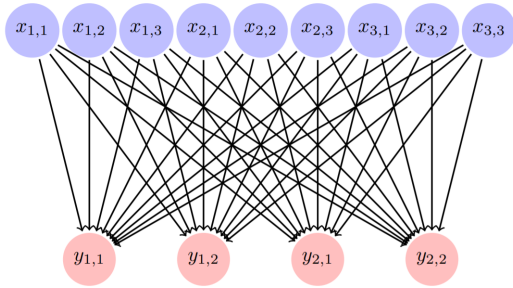


Figure 4.3: Fully connected layer [3]

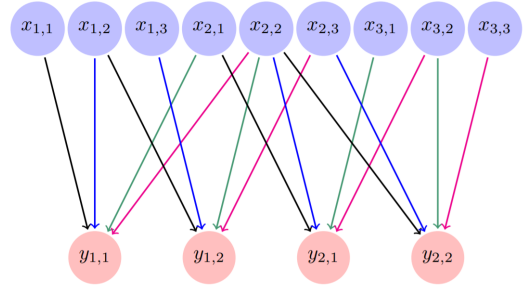


Figure 4.4: Convolutional layer [3]

- kernel size $k_1 \times k_2$.

We can consider various extra operations (see Figure 4.5):

- full padding, $o_1 = i_1 + k_1 - 1, o_2 = i_2 + k_2 - 1$
- arbitrary padding, $o = i + 2p - k + 1$, for some chosen $p > 1$
- striding, $o = \lfloor (i - k)/s \rfloor + 1$
- combinations, $o = \lfloor (i + 2p - k)/s \rfloor + 1$

With striding we shift our attention away from padding and focus on the amount by which we shift the filter. The kernel starts as usual on the leftmost part of the input, but this time it slides by steps of size s until it touches the right side of the input. Figure (4.5 c) depicts the filter position after one shift. The floor function accounts for the fact that sometimes the last possible step does not coincide with the kernel reaching the end of the input, i.e., some input units are left out.

An animated graphical representation of convolutional arithmetic can be found at https://github.com/vdumoulin/conv_arithmetic and is highly recommended.

We now turn to convolutional neural networks and their history. The difference between CNNs and FNNs as we will see, does not only lay in applying convolution. The way *Back-propagation* works in CNNs for example is also a bit different.

4.2 CNNs

This network structure was first proposed by Fukushima in 1988 [22]. It did not gain momentum however, because the hardware was not developed enough to make its computation feasible. In the 1990s backpropagation in a CNN was first used by LeCun et al. The nets that were constructed by LeCun and coworkers have the names LeNet-1 to LeNet-5 and differ in size and number of convolutions [23]. The last of those networks (LeNet-5, 1998) was actually used on an industrial scale by banks to recognise hand written digits on checks. This is basically the same application domain that we will be covering here. This architecture has become the standard ‘template’: stacking convolutions with activation function, and pooling layers (a concept that we will introduce later), and ending the network with one or more fully-connected layers. After that, researchers further improved CNNs and reported

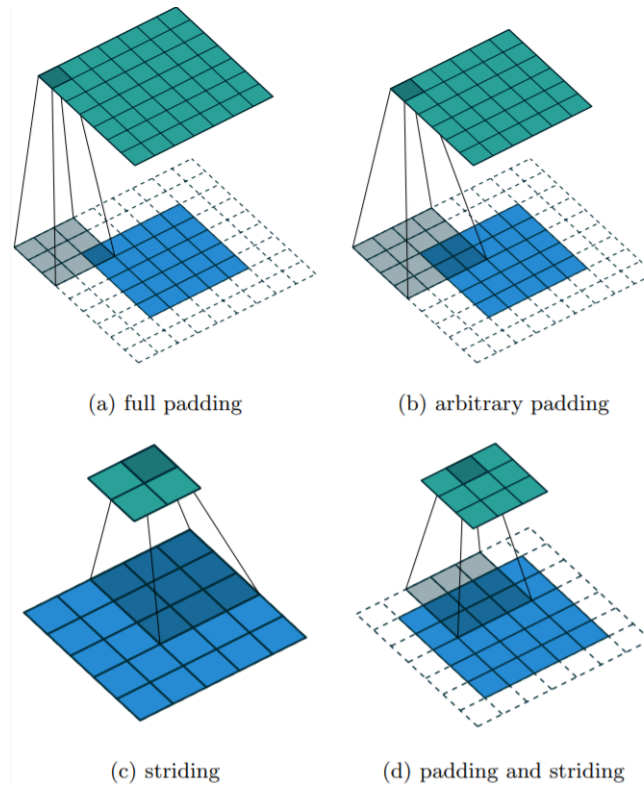


Figure 4.5: Convolution arithmetic [4]

state-of-the-art results in many recognition tasks. The success of CNNs started when they were trained on GPUs using CUDA instead of CPUs, see, e.g., Ciresan et al.[24]. The most famous CNN is AlexNet by Krizhevsky, Sutskever, and Hinton [25]. With 60M parameters, AlexNet had 8 layers — 5 convolutional and 3 fully-connected. AlexNet just stacked a few more layers onto LeNet-5. At the point of publication, the authors pointed out that their architecture was “one of the largest convolutional neural networks to date on the subsets of ImageNet”. Its novelty laid in them using a special kind of activation function called *Rectified Linear Units* (ReLUs) and in implementing *dropout*, a concept that we will also introduce later on. AlexNet became famous after winning the ILSVRC (ImageNet Large Scale Visual Recognition Competition) Challenge in 2012.

You can take a look at Figure (4.6) to see some of the examples where AlexNet was successful and some where it was not. One can observe that even in some of the cases where it did fail, the mistakes were understandable. A lot of people (including me) for example do not really know the difference between a convertible and a grille, which was still the network second best guess. CNNs continued to develop after that: OverFeat fast and OverFeat accurate by Sermanet et al. [26] 2013 won localization task in ILSVRC 2013, rank 4 for classification task, rank 1 for detection task, which were variants of AlexNet. VGG-16 by Simonyan and Zisserman [27] developed by Oxford’s Visual Geometry Group had 13 convolutional layers and was submitted to ILSVRC 2014 (weights and biases publically available). It replaces large kernels (11×11 and 5×5) in AlexNet by several small 3

Validation classification



Figure 4.6: Validation classification by AlexNet: The upper pictures (*mite*, *container ship*, *motor scooter*, and *leopard*) were classified correctly while the lower ones (*grille*, *mushroom*, *cherry*, and *Madagascar cat*) were not [5]

$\times 3$ kernels. GoogLeNet (Inception) v1 by Szegedy et al. [28] 2014 had 22 layers and was the winner of ILSVRC 2014. It Contains inception modules: stacking different types of convolutions. It uses 1×1 kernels for dimensionality reduction. ResNet by He et al. [29], ResNet-152 won ILSVRC 2015. It introduced residual connections. State-of-the-art training of ResNet-50 in 224 seconds by researchers at Sony was presented in Mikami et al. [30]. There are many more CNNs that we can not cover here. A nice and some what comprehensive overview of different CNNs and their architecture can be found at <https://towardsdatascience.com/illustrated-10-cnn-architectures-95d78ace614d>. It goes with out saying that the current trend seems to be stacking more and more layers on top of each other while trying to increase parallelism at the same time.

Convolutional neural networks are based on using kernels as weights and learning them. Starting with LeNet-5, convolutional neural networks (CNN) have typically had a standard structure – stacked convolutional layers (optionally followed by contrast normalization and maxpooling) are followed by one or more fully-connected layers. Variants of this basic design are prevalent in the image classification literature and have yielded the best results to-date on MNIST, CIFAR and most notably on the ImageNet classification challenge as mentioned before. CNNs have several advantages over fully connected ANNs, including being more similar to the human visual processing system [22], being highly optimized in structure for processing 2D and 3D images, and being effective at learning and extracting abstractions of

2D features. The max pooling layer of CNNs, is effective in absorbing shape variations and as we have mentioned already, the sparsity of the weight matrix helps accelerate the learning process. They differ from fully connected ANN in:

- Feedforwardness: We simply use convolution in place of the structured weight matrix.
- Backpropagation of the δ vectors: Although we can use the transposed structured weight matrix, we will see that there exist some better ways.
- Update of the kernels vs. known update of the weight matrices: updating a sparse block Toeplitz structured matrix by a rank-one update would most probably not result in a new structured matrix of the same type. We specify hence a better way to implement backpropagation in the context of CNNs in Section 4.2.2

4.2.1 Pooling

A convolutional layer is often followed by a pooling layer, which reduces the dimension by mapping small regions of pixels into single numbers. For example, when these small regions are taken to be squares of four neighboring pixels in a two dimensional image, a max pooling or average pooling layer replaces each set of four by their maximum or average value, respectively. Pooling is typically applied after a convolutional layer. The most common pooling layers in 2D are given by

Max pooling: mostly reduction of four neighbouring pixels to one that carries the maximal value.

Min pooling: basically like max pooling, only based on minimum.

Averaging: mostly reduction of four neighbouring pixels to one value that contains the average of the four.

Dropping: only every second, third, $\dots$ row and column is used, this is usually implemented as part of the convolution (convolution with strides) as there is no need to compute quantities that are dropped afterwards.

4.2.2 Backpropagation in CNNs

In the Feedforward step we define $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ and $\mathbf{a} = a(\mathbf{z})$ as before. The multiplication with $\mathbf{W}^T$ in the backpropagation of the δ vectors can be interpreted as a 1D and a 2D convolution, where $\mathbf{W}^T$ is given by

$$\left(\begin{array}{ccc|ccc} f_1 & & & & & \\ \vdots & \ddots & & & & \\ f_k & \dots & f_1 & & & \\ & & \ddots & \ddots & \ddots & \\ & & & f_k & \dots & f_1 \\ \hline & & & & \ddots & \vdots \\ & & & & & f_k \end{array} \right) \text{ or } \left(\begin{array}{cc|cc} f_{2,2} & 0 & 0 & 0 \\ f_{2,1} & f_{2,2} & 0 & 0 \\ 0 & f_{2,1} & 0 & 0 \\ \hline f_{1,2} & 0 & f_{2,2} & 0 \\ f_{1,1} & f_{1,2} & f_{2,1} & f_{2,2} \\ 0 & f_{1,1} & 0 & f_{2,1} \\ \hline 0 & 0 & f_{1,2} & 0 \\ 0 & 0 & f_{1,1} & f_{1,2} \\ 0 & 0 & 0 & f_{1,1} \end{array} \right)$$

respectively. This time it is the full convolution (i.e., with zero-padded input) and with flipped kernel (1D) or doubly flipped = rotated kernel (2D). In the d D case we have to pad and flip along every dimension. We now want to update the kernel. Remember we have to be careful here to make sure we maintain the sparse block Toeplitz structured matrix. We reconsider the derivation. Let $\boldsymbol{\delta} = \partial \mathbf{C} / \partial \mathbf{z}$ and $\mathbf{z} = \mathbf{a} * \mathbf{f} + \mathbf{b}$ be computed using a convolution with kernel $\mathbf{f}$. This corresponds to a more general case because we have an activated $\mathbf{a}$ instead of input $\mathbf{x}$. We write the convolution using multi-indices $\mathbf{i} = (i_1, \dots, i_d)$, $\mathbf{j} = (j_1, \dots, j_d)$, and $\mathbf{k} = (k_1, \dots, k_d)$,

$$z_i = (\mathbf{a} * \mathbf{f})_{\mathbf{i}} + b_{\mathbf{i}} = \sum_{\mathbf{j}=\mathbf{e}}^{\mathbf{k}} \mathbf{a}_{\mathbf{i}-\mathbf{j}} \mathbf{f}_{\mathbf{j}} + b_{\mathbf{i}}, \quad \mathbf{f} \in \mathbb{R}^{\mathbf{k}}, \quad \mathbf{e} = (1, \dots, 1).$$

This means that the convolution sum starts at $\mathbf{j} = \mathbf{e} = (1, \dots, 1)$ i.e. at one in every direction and keeps increasing until we hit the indices of the kernel in every dimension $\mathbf{k} = (k_1, \dots, k_d)$. Since the derivative of the *cost* function with respect to the *bias* does not differ in CNNs we turn to compute the derivative of the *cost* function with respect to $\mathbf{f}$ using the fact that we already know the derivative of the *cost* function with respect to $\mathbf{z}$. For that we will denote the flipped $\boldsymbol{\delta}$ in every dimension by $\tilde{\boldsymbol{\delta}}$, i.e. $\tilde{\delta}_{\mathbf{k}-\mathbf{i}+1} = \delta_{\mathbf{i}}$. To compute the kernel update, and since we already know how to compute (the multi-dimensional array) $\boldsymbol{\delta}$ using backpropagation, we apply convolution of the flipped $\mathbf{a}(\tilde{a}_{\mathbf{j}-\mathbf{i}} := a_{\mathbf{i}-\mathbf{j}})$ with the kernel $\boldsymbol{\delta}$,

$$\frac{\partial \mathbf{C}}{\partial f_j} = \sum_{\mathbf{i}} \frac{\partial z_{\mathbf{i}}}{\partial f_j} \frac{\partial \mathbf{C}}{\partial z_{\mathbf{i}}} = \sum_{\mathbf{i}} a_{\mathbf{i}-\mathbf{j}} \delta_{\mathbf{i}} = (\tilde{\mathbf{a}} * \boldsymbol{\delta})_{\mathbf{j}}.$$

With this we have finished the theory part and turn now to the implementation.

5 Implementation

5.1 MNIST handwritten digit dataset

The MNIST database of handwritten digits has a training set of 60,000 examples. The acronym stands for the Modified National Institute of Standards and Technology. It is a subset of a larger set available from NIST. The digits are centered in a fixed-size 28×28 pixel square image. The values of the pixels have all been normalized to be between 0 and 9. It is a good database for people who want to try learning techniques and pattern recognition methods on real-world data while spending minimal efforts on preprocessing and formatting and it is considered as a basis for learning and practicing how to develop, evaluate, and use convolutional deep learning neural networks for image classification. The original black and white images from NIST were size normalized to fit in a 20×20 pixel box while preserving their aspect ratio. The resulting images contain grey levels as a result of the anti-aliasing technique used by the normalization algorithm. The images were centered in a 28×28 image by computing the center of mass of the pixels, and translating the image so as to position this point at the center of the 28×28 field [31]. We will be using this dataset in this work. There are many places where it can be downloaded from. We will use the one from **keras.datasets**.

```
1 from keras.datasets import mnist
```

In Figure (5.1) you can see an example of several images taken from that dataset.

5.2 Network training and testing

After importing the dataset we then have to split it first into an input X and an output Y and then further split it into a train and test data sets. The function **load_dataset ()** is used to accomplish just that.

```
1
2 # load train and test dataset
3 def load_dataset():
4     # load dataset
5     (trainX, trainY), (testX, testY) = mnist.load_data()
6     # reshape dataset to have a single channel
7     trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
8     testX = testX.reshape((testX.shape[0], 28, 28, 1))
9     # one hot encode target values
10    trainY = to_categorical(trainY)
11    testY = to_categorical(testY)
12    return trainX, trainY, testX, testY
```

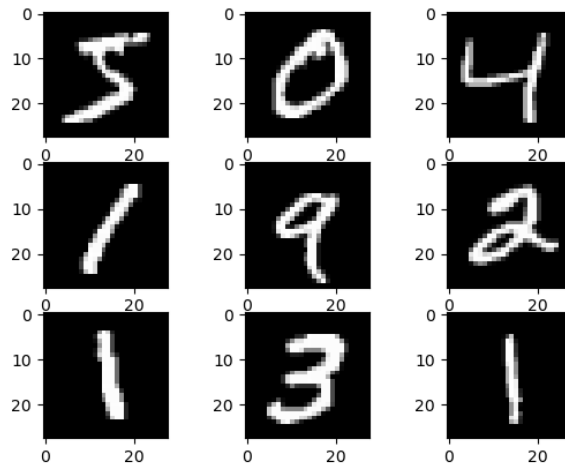


Figure 5.1: Plot of a Subset of Images From the MNIST Dataset

Most neural networks face the problem of overfitting. Which means that quite often the ML model learns to accurately predict data it is trained on, but fails to generalize to other new data. This is doomed to happen if the data that we are testing on is the same one we trained on. This is why the split of data into training and testing sets was necessary. This can be enhanced by further splitting the training set into a train and a validation dataset. If the dataset available is small and it is split into train and test sets, there will not be enough data in the training dataset for the model to learn an effective mapping of inputs to outputs. There will also not be enough data in the test set to effectively evaluate the model performance. The estimated performance could be overly optimistic or overly pessimistic. Hence, we need to separate our input data into training, validation, and testing subsets to prevent the model from overfitting and to evaluate the model effectively. For the validation part we will use *k-fold Cross-validation*, which is a statistical method used to estimate the skill of machine learning models. The parameter k refers to the number of groups that a given data sample is to be split into. The general procedure is as follows:

1. Shuffle the dataset randomly.
2. Split the dataset into k groups
3. For each unique group:
 1. Take the group as a hold out or test data set.
 2. Take the remaining groups as a training data set.
 3. Fit a model on the training set and evaluate it on the test set.
 4. Retain the evaluation score and discard the model.
4. Summarize the skill of the model using the sample of model evaluation scores.

The results of a *k-fold cross-validations* run are often summarized with the mean of the model skill scores. The **KFold()** scikit-learn class can be used to provide an implementation that will split a given data sample up. It takes as arguments the number of splits, whether or not to shuffle the sample, and the seed for the pseudorandom number generator used

prior to the shuffle. The `split()` function can then be called on the class where the data sample is provided as an argument. Called repeatedly, the split will return each group of train and test sets. Specifically, arrays are returned containing the indexes into the original data sample of observations to use for train and test sets on each iteration. We will train the model for a modest 10 training epochs with a default batch size of 32 examples. The test set for each fold will be used to evaluate the model both during each epoch of the training run, so that we can later create learning curves, and at the end of the run, so that we can estimate the performance of the model. As such, we will keep track of the resulting history from each run, as well as the classification accuracy of the fold.

The `evaluate_model()` function below implements these behaviors, taking the training dataset as arguments and returning a list of accuracy scores and training histories that can be later summarized.

```

1 # evaluate a model using k-fold cross-validation
2 def evaluate_model(dataX, dataY, n_folds=5):
3     scores, histories = list(), list()
4     # prepare cross validation
5     kfold = KFold(n_folds, shuffle=True, random_state=1)
6     # enumerate splits
7     for train_ix, test_ix in kfold.split(dataX):
8         # define model
9         model = define_model()
10        # select rows for train and test
11        trainX, trainY, testX, testY = dataX[train_ix], dataY[train_ix], dataX[
12            test_ix], dataY[test_ix]
13        # fit model
14        history = model.fit(trainX, trainY, epochs=10, batch_size=32,
15            validation_data=(testX, testY), verbose=0)
16        # evaluate model
17        _, acc = model.evaluate(testX, testY, verbose=0)
18        print('> %.3f' % (acc * 100.0))
19        # stores scores
20        scores.append(acc)
21        histories.append(history)
22    return scores, histories

```

Finally we turn to defining the learning model, which turns out to be the easiest part to code largely due to the help of libraries such as *TensorFlow* from *Keras* developed by *Google* and *PyTorch* developed by *Facebook*. Since we already used *Keras* to import our data set we are going to stick here with *TensorFlow*. To define the model, two main aspects need to be considered. The first one is the convolutional and pooling layers, which comprises the feature extraction front end. Those two already have an implementation that can be imported from `keras.layers`. We choose a small filter size (3,3) and a modest number of filters (32) followed by a max pooling layer for each (2,2) patch of the feature map. The filter maps can then be flattened to provide features to the classifier. The second aspect is the dense or fully connected layers, which works as the classifier backend. We need 10 nodes in the last layer in order to predict the 10 different classes. Between the output and the pooling layer we add a dense (fully connected) layer with 100 nodes.

All layers will use the ReLU activation function and the He weight initialization scheme, both best practices. An initialization scheme is a method that specifies how to assign the starting values of the parameters that need to be trained. We will use a conservative configuration for the stochastic gradient descent optimizer with a learning rate of 0.01. As a loss function we will optimize over the so called *categorical cross-entropy* loss function, suitable for multi-class classification [32].

```

1 # define cnn model
2 def define_model():
3     model = Sequential()
4     model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='
        he_uniform', input_shape=(28, 28, 1)))
5     model.add(MaxPooling2D((2, 2)))
6     model.add(Flatten())
7     model.add(Dense(100, activation='relu', kernel_initializer='he_uniform')
8     )
9     model.add(Dense(10, activation='softmax'))
10    # compile model
11    opt = SGD(lr=0.01, momentum=0.9)
12    model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['
        accuracy'])
13    return model

```

The number of layers and the size of each layer are part of what is called hyperparameters in machine learning. Those are the parameters whose value is used to control the learning process and are set before the learning begins. By contrast, the values of the other (normal) parameters are derived via training. In the next section we will explore applying different hyperparameters and the effect of that on the result. A complete implementation can be found at <https://github.com/HaniTUHH/Convolutional-Networks> or with a more thorough explanation at <https://machinelearningmastery.com/how-to-develop-a-convolutional-neural-network-from-scratch-for-mnist-handwritten-digit-classification/>

5.3 Results

A run of the previous implantation with its stated hyperparameters results in an average accuracy of 98.6 percent. In Figure (5.2) one can observe both the improvement in accuracy as well as the minimization of the loss function as the learning proceeds for our ML model. The blue lines indicate model performance on the training dataset while the orange lines indicate performance on the hold out test dataset. In this case, we can see that the model generally achieves a good fit, with train and test learning curves converging. There is no obvious sign of over- or underfitting [33]. In Figure (5.3) one can take a look at 3 examples where the network failed to predict accurately. Same as in (4.6) we can observe here that the some of the miss-classifications like 1 and 2 would have probably also been miss-classified by a human classifier. For the third example however, it seems that fact that the digit was tilted mystified the model when that would not have been a problem for a human classifier

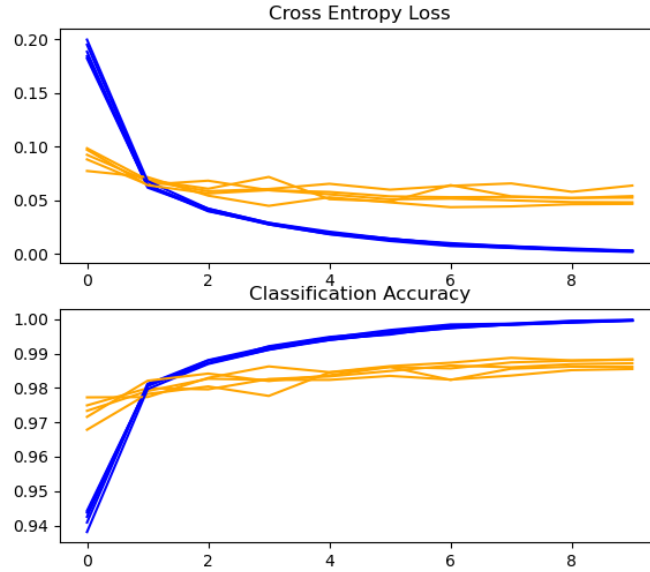


Figure 5.2: Loss and accuracy learning curves for the model during k-Fold cross-validation

To make sure that the hyperparameters we picked are appropriate several test runs were

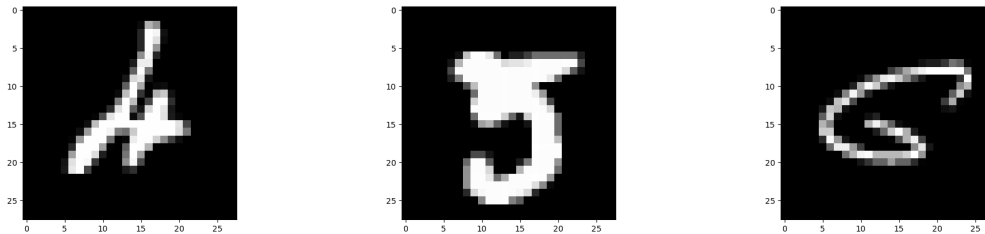


Figure 5.3: Three wrongly classified images from the MNIST dataset. They are in order: 4, 5 and 6

performed. Our subject of interest was to measure how would a deeper neural network preform when the increase in depth layed in increasing the convolutional feature extraction part of the network vs increasing the depth of the classifier backend. To do that, in one test run we added two convolutional layers before the first dense layer, while in the other one we add two dense layers before the output layer. We observe that while adding two additional dense layers did not deliver any improvements in accuracy (98.6) adding two convolutional layers did slightly improve the accuracy (98.9) while not giving any sign of over fitting. The modified code of the two test runs along with the learning curves can also be found at <https://github.com/HaniTUHH/Convolutional-Networks>

6 Conclusion and Future Work

6.1 Summary

In this work we went through the fundamentals of *deep learning* defining *mathematical neurons*, *activation function*, and *artificial neural networks*. We went from there to define *Backpropagation* through first explaining what are the *cost* function and the *gradient descent* method and how can they be applied to facilitate *Backpropagation* such that learning can occur. We built on that to introduce the notion of *convolution* and its usefulness in feature/pattern extraction and explained how it can be applied in neural networks. We went through the little differences between backpropagation in FNNs vs in CNNs. We specified the importance of CNNs in image classification and elaborated on their history as well as their advantage in this field in comparison with other machine learning models. We gave near the end a walk through of an implementation written in python. We wrapped that up with two little test runs where we deepened the network one time in the frontend and one time in the backend and concluded with modest confidence that deepening the convolutional layers seems to be more advantageous than deepening the fully connected layers if every thing else is held constant. It remains to be settled if this hold true for every other image classification problem. We conclude, this time with a relatively high confidence, that the MNIST hand written digit classification problem is effectively 'solved' due to our work here and various other publications [34] .

Bibliography

- [1] KRIESEL, David: *Ein kleiner Überblick über Neuronale Netze*. 2007 <http://www.dkriesel.com>
- [2] HIGHAM, Catherine F. ; HIGHAM, Desmond J.: *Deep Learning: An Introduction for Applied Mathematicians*. MIT Press, 2016. – <https://epubs.siam.org/doi/pdf/10.1137/18M1165748>
- [3] ZEMKE, Jens-Peter M.: *Mathematics of Neural Networks*. Lecture Notes, TUHH, 2021
- [4] DUMOULIN, Vincent ; VISIN, Francesco: *A guide to convolution arithmetic for deep learning*. 2018
- [5] KRIZHEVSKY, Alex ; SUTSKEVER, Ilya ; HINTON, Geoffrey E.: ImageNet Classification with Deep Convolutional Neural Networks. In: *Commun. ACM* 60 (2017), Mai, Nr. 6, 84–90. <http://dx.doi.org/10.1145/3065386>. – DOI 10.1145/3065386. – ISSN 0001–0782
- [6] FUJIYOSHI, Hironobu ; HIRAKAWA, Tsubasa ; YAMASHITA, Takayoshi: Deep learning-based image recognition for autonomous driving. In: *IATSS Research* 43 (2019), Nr. 4, 244–252. <http://dx.doi.org/https://doi.org/10.1016/j.iatssr.2019.11.008>. – DOI <https://doi.org/10.1016/j.iatssr.2019.11.008>. – ISSN 0386–1112
- [7] BRINDHA, P G. ; KAVINRAJ, M ; MANIVASAKAM, P ; PRASANTH, P: Brain tumor detection from MRI images using deep learning techniques. In: *IOP Conference Series: Materials Science and Engineering* 1055 (2021), feb, Nr. 1, 012115. <http://dx.doi.org/10.1088/1757-899x/1055/1/012115>. – DOI 10.1088/1757-899x/1055/1/012115
- [8] MCCULLOCH, W. S. ; PITTS, W.: “A logical calculus of the ideas immanent in nervous activity. *Bull. Math. Biophys.* 5, 1943. – 115–133 S. <http://dx.doi.org/10.1007/bf02478259>. <http://dx.doi.org/10.1007/bf02478259>. ISSN 0007–4985
- [9] HEBB, D.O.: *The Organization of Behavior: A Neuropsychological Theory*. Wiley, 1949 (Neuropsychological Theory). <https://books.google.de/books?id=dZ0eDiLTWuEC>. – ISBN 9780471367277
- [10] KLEENE, SC: *Representations of events in nerve and finite automata. Automata Studies*. 1956

-
- [11] ROSENBLATT, F.: *The Perceptron, a Perceiving and Recognizing Automaton Project Para.* Cornell Aeronautical Laboratory, 1957 (Report: Cornell Aeronautical Laboratory). https://books.google.de/books?id=P_XGpGAACAAJ
- [12] RUMELHART, David E. ; HINTON, Geoffrey E. ; WILLIAMS, Ronald J.: Learning Representations by Back-propagating Errors. In: *Nature* 323 (1986), Nr. 6088, 533–536. <http://dx.doi.org/10.1038/323533a0>. – DOI 10.1038/323533a0
- [13] KELLEY, HENRY J.: Gradient Theory of Optimal Flight Paths. In: *ARS Journal* 30 (1960), Nr. 10, 947-954. <http://dx.doi.org/10.2514/8.5282>. – DOI 10.2514/8.5282
- [14] BRYSON, Arthur ; HO, Y.-C ; SIOURIS, George: Applied Optimal Control: Optimization, Estimation, and Control. In: *Systems, Man and Cybernetics, IEEE Transactions on* 9 (1979), 07, S. 366 – 367. <http://dx.doi.org/10.1109/TSMC.1979.4310229>. – DOI 10.1109/TSMC.1979.4310229
- [15] KVAAL, Knut ; MCEWAN, Jean A.: Analysing Complex Sensory Data by Non-Linear Artificial Neural Networks. Version: 1996. [http://dx.doi.org/https://doi.org/10.1016/S0922-3487\(96\)80028-1](http://dx.doi.org/https://doi.org/10.1016/S0922-3487(96)80028-1). In: NAES, Tormod (Hrsg.) ; RISVIK, Einar (Hrsg.): *Multivariate analysis of data in sensory science* Bd. 16. Elsevier, 1996. – DOI [https://doi.org/10.1016/S0922-3487\(96\)80028-1](https://doi.org/10.1016/S0922-3487(96)80028-1). – ISSN 0922-3487, 103-133
- [16] FAHLMAN, Scott E.: An empirical study of learning speed in back-propagation networks. 1988. – Forschungsbericht
- [17] TOLLENAERE, Tom: SuperSAB: Fast adaptive back propagation with good scaling properties. In: *Neural Networks* 3 (1990), Nr. 5, 561-573. [http://dx.doi.org/https://doi.org/10.1016/0893-6080\(90\)90006-7](http://dx.doi.org/https://doi.org/10.1016/0893-6080(90)90006-7). – DOI [https://doi.org/10.1016/0893-6080\(90\)90006-7](https://doi.org/10.1016/0893-6080(90)90006-7). – ISSN 0893-6080
- [18] RIEDMILLER, M. ; BRAUN, H.: A direct adaptive method for faster backpropagation learning: the RPROP algorithm. In: *IEEE International Conference on Neural Networks*, 1993, S. 586–591 vol.1
- [19] KAWANAKA, Yoshiya ; AIYOSHI, Eitaro: Using Methods of the Discrete Neural Networks for Nonlinear Optimization Problems and Their Applications to the Learning Algorithm of Neural Networks. In: *Transactions of the Society of Instrument and Control Engineers* 31 (1995), Nr. 2, S. 244–253. <http://dx.doi.org/10.9746/sicetr1965.31.244>. – DOI 10.9746/sicetr1965.31.244
- [20] KRAMER, Alan ; SANGIOVANNI-VINCENTELLI, Alberto: Efficient Parallel Learning Algorithms for Neural Networks. In: TOURETZKY, D. (Hrsg.): *Advances in Neural Information Processing Systems* Bd. 1, Morgan-Kaufmann, 1989
- [21] BRUST, Clemens-Alexander ; SICKERT, Sven ; SIMON, Marcel ; RODNER, Erik ; DENZLER, Joachim: Neither Quick Nor Proper - Evaluation of QuickProp for Learning Deep

- Neural Networks. In: *CoRR* abs/1606.04333 (2016). <http://arxiv.org/abs/1606.04333>
- [22] FUKUSHIMA, Kunihiko ; MIYAKE, Sei: Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Visual Pattern Recognition. In: AMARI, Shun-ichi (Hrsg.) ; ARBIB, Michael A. (Hrsg.): *Competition and Cooperation in Neural Nets*. Berlin, Heidelberg : Springer Berlin Heidelberg, 1982. – ISBN 978-3-642-46466-9, S. 267–285
- [23] LECUN, Y. ; BOTTOU, L. ; BENGIO, Y. ; HAFFNER, P.: Gradient-based learning applied to document recognition. In: *Proceedings of the IEEE* 86 (1998), Nr. 11, S. 2278–2324. <http://dx.doi.org/10.1109/5.726791>. – DOI 10.1109/5.726791
- [24] CIRESAN, Dan ; MEIER, Ueli ; MASCI, Jonathan ; GAMBARDELLA, Luca M. ; SCHMIDHUBER, Jürgen: Flexible, High Performance Convolutional Neural Networks for Image Classification., 2011, S. 1237–1242
- [25] KRIZHEVSKY, Alex ; SUTSKEVER, Ilya ; HINTON, Geoffrey E.: ImageNet Classification with Deep Convolutional Neural Networks, Curran Associates, Inc., 2012
- [26] SERMANET, Pierre ; EIGEN, David ; ZHANG, Xiang ; MATHIEU, Michael ; FERGUS, Rob ; LECUN, Yann: OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks. (2013), 12
- [27] SIMONYAN, Karen ; ZISSERMAN, Andrew: Very Deep Convolutional Networks for Large-Scale Image Recognition. In: *arXiv 1409.1556* (2014), 09
- [28] SZEGEDY, Christian ; LIU, Wei ; JIA, Yangqing ; SERMANET, Pierre ; REED, Scott ; ANGUELOV, Dragomir ; ERHAN, Dumitru ; VANHOUCKE, Vincent ; RABINOVICH, Andrew: Going deeper with convolutions. In: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, S. 1–9
- [29] HE, Kaiming ; ZHANG, Xiangyu ; REN, Shaoqing ; SUN, Jian: Deep Residual Learning for Image Recognition. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, S. 770–778
- [30] MIKAMI, Hiroaki ; SUGANUMA, Hisahiro ; U.-CHUPALA, Pongsakorn ; TANAKA, Yoshiki ; KAGEYAMA, Yuichi: ImageNet/ResNet-50 Training in 224 Seconds. In: *CoRR* abs/1811.05233 (2018). <http://arxiv.org/abs/1811.05233>
- [31] https://en.wikipedia.org/wiki/MNIST_database
- [32] <https://machinelearningmastery.com/cross-entropy-for-machine-learning/>
- [33] <https://towardsdatascience.com/learning-curve-to-identify-overfitting-underfitting-problems-133177f38df5>
- [34] VAFADAR, Majid: *A Convolutional Neural Network solution for MNIST dataset*. 02 2018